AI

WORD SEARCH GENERATOR AND SOLVER

Documentation and Project by Jomana Shaltout

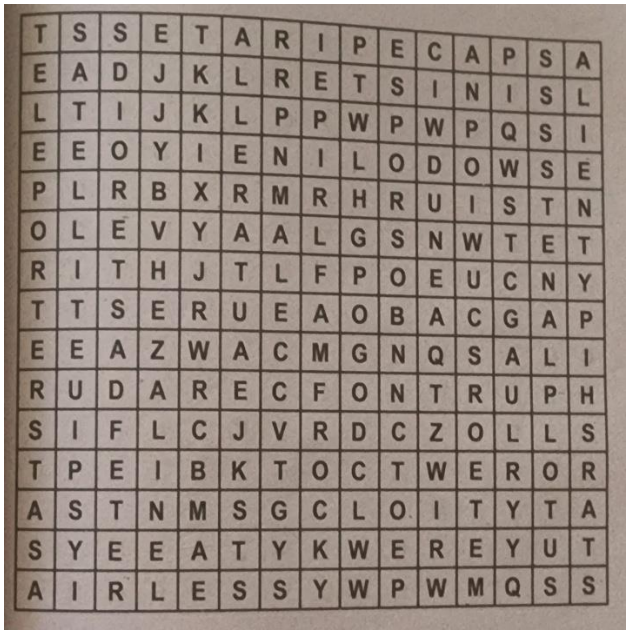
Contents

Overview

This project is inspired by the 'Dreamland Super Word Search, Ultimate Word Search with Solutions' book series of 16 books.

Project Plan

In this project, the program is supposed to generate a grid, fill it up with the puzzle words, then fill the remaining empty spaces with fill letters. Afterwards, the user will enter the word they want to find, and the program will proceed to look for it. This project is split into three separate files: Generator, Solver, and GUI. Below is an image display of how the grid should look like:



T	S	S	E	T	A	R	I	P	E	C	A	P	S	A
E	A	D	J	K	L	R	E	T	S	I	N	I	S	L
L	T	I	J	K	L	P	P	W	P	W	P	Q	S	I
E	E	O	Y	I	E	N	I	L	O	D	O	W	S	E
P	L	R	B	X	R	M	R	H	R	U	I	S	T	N
O	L	E	V	Y	A	A	L	G	S	N	W	T	E	T
R	I	T	H	J	T	L	F	P	O	E	U	C	N	Y
T	T	S	E	R	U	E	A	O	B	A	C	G	A	P
E	E	A	Z	W	A	C	M	G	N	Q	S	A	L	I
R	U	D	A	R	E	C	F	O	N	T	R	U	P	H
S	I	F	L	C	J	V	R	D	C	Z	O	L	L	S
T	P	E	I	B	K	T	O	C	T	W	E	R	O	R
A	S	T	N	M	S	G	C	L	O	I	T	Y	T	A
S	Y	E	E	A	T	Y	K	W	E	R	E	Y	U	T
A	I	R	L	E	S	S	Y	W	P	W	M	Q	S	S

Figure 1: Grid structure from the books.

The grid is 15x15, where each cell will hold a letter. This letter could be from a word or could be a random fill letter.

Concepts in the Project

There are several concepts used to make this project. 2D lists, Vectors, Bound Checking, Limited Backtracking, String/List Conversions, Randomisation, Collision Detection, Linear Search, and Result Tracking.

- 2D lists are essentially a list of lists. This will be used to make the grid.
- Vectors are used in this project to symbolise the directions. There are 8 possible directions in which a word can be found in a word search puzzle; right, left, down, up, down-right diagonal, down-left diagonal, up-right diagonal, and up-left diagonal.
- Bound Checking is used to verify that the rows and columns of the grid do not exceed 15 (index 0 to 14). This helps in avoiding index errors.

- Limited Backtracking is used when the words are placed. A word may not fit in the chosen spot, so it retries placing it in a different random location until it fits. It is 'limited' because instead of undoing all the previously placed words, we are only retrying the word that does not fit.
- String/List conversions were done implicitly during the grid creation, and the accessing of individual characters.
- Positions, directions, and fill letters were randomised.
- Conflicts can occur when a cell is occupied by a different letter. We detect these conflicts and try placing our word elsewhere.
- For each cell on the grid, there are 8 directions. We walk through each direction to see I the word is there.
- Found words are stored to track the results.

Since we split our working to three files, we will focus on each alone:

Files	Features
puzzleGenerator	There are two lists present in the file, one list holds the possible directions a word can be found, and the other list holds letters A-Z so that they can be used to randomly fill the empty spaces of the grid. A grid was created using a 2D list (a list that contains a list of columns and a list of rows) Using randint() from the random library, we randomised the choice of one row and one column (a cell) in which the first letter will be placed, then using choice() from the random library, the directions were randomised. After a direction is chosen, the word is filled in that direction. After all words are placed, using choice(), random letters are chosen to fill in the remaining cells.
puzzleSolver	To find the words, we will iterate over all the cells in the grid and check if the current cell holds required index letter or not. If it does, the cell coordinates are returned, otherwise it will return not found. This is done with Linear Search A function was created to check if the current index of the letter matches the letter at the current cell, then choose a direction to search in. If the next index matches in that direction it keeps going in the same direction until proven otherwise.

puzzleGUI	There are various frames that the GUI switches between flawlessly. The main screen, the puzzle gallery screen, and the puzzle screen. There are buttons for navigation between the screens, and a title to tell you which screen you're on.

GUI

The GUI was made using the tkinter library. Below are the wireframes of each screen:

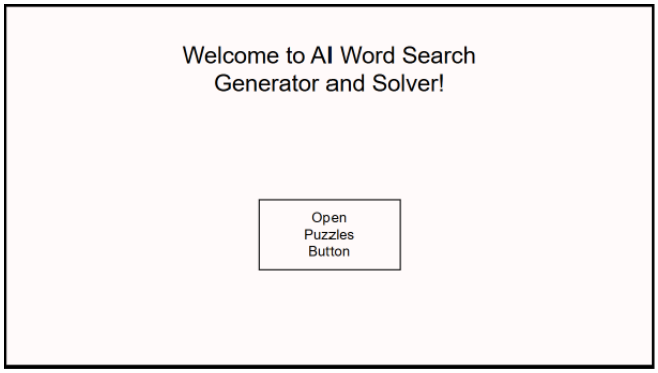


Figure 2: Main Screen Wireframe

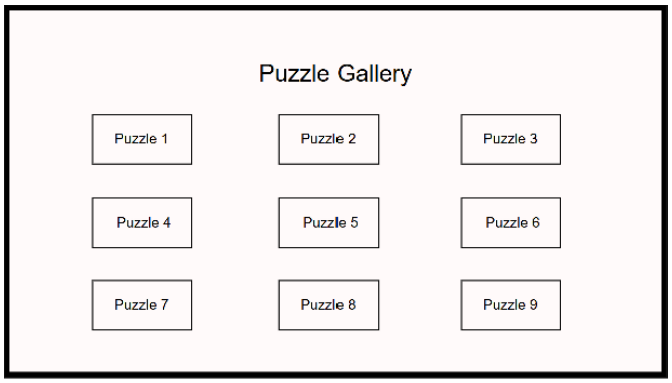


Figure 3: Puzzle Gallery Screen Wireframe

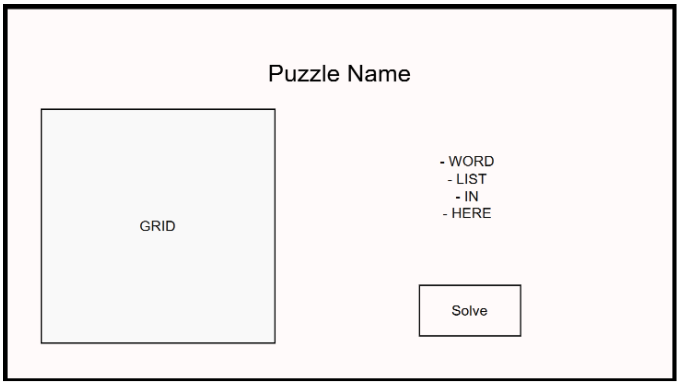


Figure 4: Puzzle Screen Wireframe

Using these wireframes, we were able to establish the number of frames needed. To switch between them, each screen had a main frame and to switch from one frame to the other we used pack_forget() on the current frame then pack() to make the wanted frame appear.

In Figure 3, this frame had a sub-frame inside it. This frame held a 3x3 area of buttons, where each button led to its respective puzzle. In Figure 4, the frame held 3 sub-frames inside. A right frame, left frame, and top frame, to allow for easy padding and display. The right frame held the word list, the entry box where the user enters the word they want to look for, and the solve button. The left frame held the generated grid, and the top frame held the puzzle title.

Limitations

As I've tested this project multiple times, I've noticed some limitations that are worth mentioning:

- Most short words (three-letter words) will not work out as they could be found accidentally within the random letter placing rather than finding them where they were placed by the generator.
- Words with spaces and signs (like a hyphen, for example) will not work as the space will not be skipped like how it's done in normal word search.
- Sometimes, when generating the grid, one or two words may not end up being placed. But that happens rarely.
- The grid generation is not fixed. Every time you load the same puzzle, the generation changes the word placement, and the random fill words.

Conclusion

This project successfully demonstrates how fundamental programming concepts can be combined to build an interactive and visually engaging application. The project brings together grid-based data structures, randomised word placement, directional search, and a fully functional GUI – all built using built-in Python libraries.

The generator places words across 8 directions in a 15x15 grid, the solver efficiently locates the words using linear search, and the GUI provides a smooth experience for the user. There are features like animated highlighting and word discovery, and strikethrough feedback for when a word is found.

Overall, this project serves as a strong foundation for understanding how algorithms and user interfaces work together and could be extended by adding features like a timer, or the ability for users to create their own puzzles.